

CPU Scheduling-FCFS

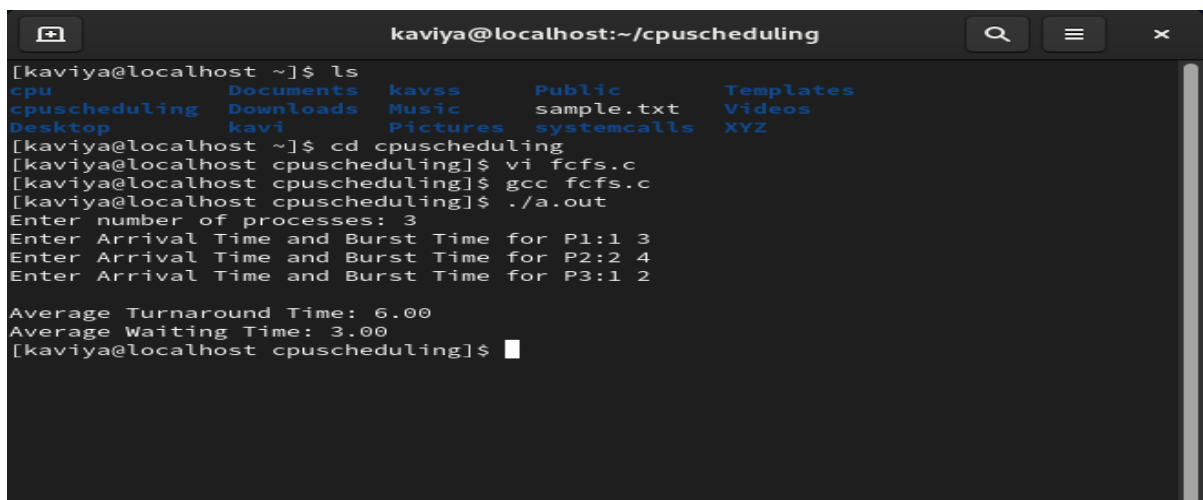
Input:

```
#include<stdio.h>
int main(){
    int n,i,j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n];
    for(i=0;i<n;i++){
        printf("Enter Arrival Time and Burst Time for P%d:", i+1);
        scanf("%d%d", &at[i], &bt[i]);
    }
    float total_tat=0, total_wt=0;
    ct[0]=at[0]+bt[0];
    for(i=1;i<n;i++){
        if(ct[i-1]<at[i])
            ct[i] =at[i]+bt[i];
        else
            ct[i]= ct[i-1]+bt[i];
    }
    for(i=0;i<n;i++){
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        total_tat+=tat[i];
        total_wt+=wt[i];
    }
    printf("\nAverage Turnaround Time: %.2f", total_tat/n);
    printf("\nAverage Waiting Time: %.2f\n", total_wt/n);
    return 0;
}
```

3,10-17

All

Output:



```
kaviya@localhost:~/cpuscheduling

[kaviya@localhost ~]$ ls
cpu          Documents  kavss      Public      Templates
cpuscheduling Downloads  Music      sample.txt  Videos
Desktop      kavi      Pictures   systemcalls XYZ
[kaviya@localhost ~]$ cd cpuscheduling
[kaviya@localhost cpuscheduling]$ vi fcfs.c
[kaviya@localhost cpuscheduling]$ gcc fcfs.c
[kaviya@localhost cpuscheduling]$ ./a.out
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1:1 3
Enter Arrival Time and Burst Time for P2:2 4
Enter Arrival Time and Burst Time for P3:1 2

Average Turnaround Time: 6.00
Average Waiting Time: 3.00
[kaviya@localhost cpuscheduling]$
```

SJF SCHEDULING ALGORITHM

```
Activities Terminal Mar 25 00:44
kaviya@localhost: ~-scheduling

kaviya@localhost:~-scheduling$ gcc sjf.c
kaviya@localhost:~-scheduling$ ./a.out
Enter number of processes: 4
kaviya@localhost:~-scheduling$ cd scheduling
kaviya@localhost:~-scheduling$ vi sjf.c
#include <stdio.h>

int main(){
    int n;
    printf("Enter number of processes: ");
    scanf("%d",&n);

    int arr[n], bt[n], ct[n], tat[n], wt[n];
    int done[n];

    for(int i=0; i<n; i++){
        printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
        scanf("%d %d",&arr[i], &bt[i]);
        done[i] = 0;
    }

    int completed = 0;
    int completionTime = 0;
    float avgTat = 0, avgWt = 0;

    while(completed < n){
        int idx = -1;
        int ardes = 60000;

        for(int i=0; i<n; i++){
            if(arr[i] < completionTime && done[i] == 0){
                if(arr[i] < ardes){
                    ardes = arr[i];
                    idx = i;
                }
            }
        }

        if(idx == -1){
            printf("Error: No process found to execute\n");
            return 1;
        }

        completionTime = arr[idx];
        completionTime += bt[idx];
        tat[idx] = completionTime - arr[idx];
        wt[idx] = tat[idx] - bt[idx];

        done[idx] = 1;
        completed++;
    }

    for(int i=0; i<n; i++){
        avgTat += tat[i];
        avgWt += wt[i];
    }

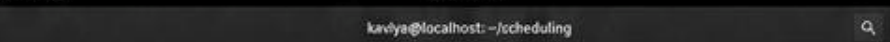
    printf("Average Turnaround Time: %.2f\n", avgTat/n);
    printf("Average Waiting Time: %.2f\n", avgWt/n);
}
```

Output

```
kaviya@localhost:~-scheduling$ ./a.out
Enter number of processes: 4
Enter Arrival and Burst Time for P1: 1 3
Enter Arrival and Burst Time for P2: 2 4
Enter Arrival and Burst Time for P3: 1 2
Enter Arrival and Burst Time for P4: 4 4

Average Turnaround Time: 6.25
Average Waiting Time: 3.00
```

PRIORITY ALGORITHM



The screenshot shows a terminal window with the title bar "Activities Terminal" and a date/time indicator "Mar 25 00:07". The prompt is "kaviya@localhost: ~/scheduling". The terminal output is as follows:

```
kaviya@localhost: ~/scheduling
[keviye@localhost ~]$ ls
bin: ca  cat  gcc  gcc-4.8.4  gcc-4.9.2  gcc-5.4.0  gcc-5.5.0  gcc-5.6.0  gcc-5.6.1  gcc-5.6.2  gcc-5.6.3  gcc-5.6.4  gcc-5.6.5  gcc-5.6.6  gcc-5.6.7  gcc-5.6.8  gcc-5.6.9  gcc-5.7.0  gcc-5.7.1  gcc-5.7.2  gcc-5.7.3  gcc-5.7.4  gcc-5.7.5  gcc-5.7.6  gcc-5.7.7  gcc-5.7.8  gcc-5.7.9  gcc-5.8.0  gcc-5.8.1  gcc-5.8.2  gcc-5.8.3  gcc-5.8.4  gcc-5.8.5  gcc-5.8.6  gcc-5.8.7  gcc-5.8.8  gcc-5.8.9  gcc-5.9.0  gcc-5.9.1  gcc-5.9.2  gcc-5.9.3  gcc-5.9.4  gcc-5.9.5  gcc-5.9.6  gcc-5.9.7  gcc-5.9.8  gcc-5.9.9  gcc-6.0.0  gcc-6.0.1  gcc-6.0.2  gcc-6.0.3  gcc-6.0.4  gcc-6.0.5  gcc-6.0.6  gcc-6.0.7  gcc-6.0.8  gcc-6.0.9  gcc-6.1.0  gcc-6.1.1  gcc-6.1.2  gcc-6.1.3  gcc-6.1.4  gcc-6.1.5  gcc-6.1.6  gcc-6.1.7  gcc-6.1.8  gcc-6.1.9  gcc-6.2.0  gcc-6.2.1  gcc-6.2.2  gcc-6.2.3  gcc-6.2.4  gcc-6.2.5  gcc-6.2.6  gcc-6.2.7  gcc-6.2.8  gcc-6.2.9  gcc-6.3.0  gcc-6.3.1  gcc-6.3.2  gcc-6.3.3  gcc-6.3.4  gcc-6.3.5  gcc-6.3.6  gcc-6.3.7  gcc-6.3.8  gcc-6.3.9  gcc-6.4.0  gcc-6.4.1  gcc-6.4.2  gcc-6.4.3  gcc-6.4.4  gcc-6.4.5  gcc-6.4.6  gcc-6.4.7  gcc-6.4.8  gcc-6.4.9  gcc-6.5.0  gcc-6.5.1  gcc-6.5.2  gcc-6.5.3  gcc-6.5.4  gcc-6.5.5  gcc-6.5.6  gcc-6.5.7  gcc-6.5.8  gcc-6.5.9  gcc-6.6.0  gcc-6.6.1  gcc-6.6.2  gcc-6.6.3  gcc-6.6.4  gcc-6.6.5  gcc-6.6.6  gcc-6.6.7  gcc-6.6.8  gcc-6.6.9  gcc-6.7.0  gcc-6.7.1  gcc-6.7.2  gcc-6.7.3  gcc-6.7.4  gcc-6.7.5  gcc-6.7.6  gcc-6.7.7  gcc-6.7.8  gcc-6.7.9  gcc-6.8.0  gcc-6.8.1  gcc-6.8.2  gcc-6.8.3  gcc-6.8.4  gcc-6.8.5  gcc-6.8.6  gcc-6.8.7  gcc-6.8.8  gcc-6.8.9  gcc-6.9.0  gcc-6.9.1  gcc-6.9.2  gcc-6.9.3  gcc-6.9.4  gcc-6.9.5  gcc-6.9.6  gcc-6.9.7  gcc-6.9.8  gcc-6.9.9  gcc-7.0.0  gcc-7.0.1  gcc-7.0.2  gcc-7.0.3  gcc-7.0.4  gcc-7.0.5  gcc-7.0.6  gcc-7.0.7  gcc-7.0.8  gcc-7.0.9  gcc-7.1.0  gcc-7.1.1  gcc-7.1.2  gcc-7.1.3  gcc-7.1.4  gcc-7.1.5  gcc-7.1.6  gcc-7.1.7  gcc-7.1.8  gcc-7.1.9  gcc-7.2.0  gcc-7.2.1  gcc-7.2.2  gcc-7.2.3  gcc-7.2.4  gcc-7.2.5  gcc-7.2.6  gcc-7.2.7  gcc-7.2.8  gcc-7.2.9  gcc-7.3.0  gcc-7.3.1  gcc-7.3.2  gcc-7.3.3  gcc-7.3.4  gcc-7.3.5  gcc-7.3.6  gcc-7.3.7  gcc-7.3.8  gcc-7.3.9  gcc-7.4.0  gcc-7.4.1  gcc-7.4.2  gcc-7.4.3  gcc-7.4.4  gcc-7.4.5  gcc-7.4.6  gcc-7.4.7  gcc-7.4.8  gcc-7.4.9  gcc-7.5.0  gcc-7.5.1  gcc-7.5.2  gcc-7.5.3  gcc-7.5.4  gcc-7.5.5  gcc-7.5.6  gcc-7.5.7  gcc-7.5.8  gcc-7.5.9  gcc-7.6.0  gcc-7.6.1  gcc-7.6.2  gcc-7.6.3  gcc-7.6.4  gcc-7.6.5  gcc-7.6.6  gcc-7.6.7  gcc-7.6.8  gcc-7.6.9  gcc-7.7.0  gcc-7.7.1  gcc-7.7.2  gcc-7.7.3  gcc-7.7.4  gcc-7.7.5  gcc-7.7.6  gcc-7.7.7  gcc-7.7.8  gcc-7.7.9  gcc-7.8.0  gcc-7.8.1  gcc-7.8.2  gcc-7.8.3  gcc-7.8.4  gcc-7.8.5  gcc-7.8.6  gcc-7.8.7  gcc-7.8.8  gcc-7.8.9  gcc-7.9.0  gcc-7.9.1  gcc-7.9.2  gcc-7.9.3  gcc-7.9.4  gcc-7.9.5  gcc-7.9.6  gcc-7.9.7  gcc-7.9.8  gcc-7.9.9  gcc-8.0.0  gcc-8.0.1  gcc-8.0.2  gcc-8.0.3  gcc-8.0.4  gcc-8.0.5  gcc-8.0.6  gcc-8.0.7  gcc-8.0.8  gcc-8.0.9  gcc-8.1.0  gcc-8.1.1  gcc-8.1.2  gcc-8.1.3  gcc-8.1.4  gcc-8.1.5  gcc-8.1.6  gcc-8.1.7  gcc-8.1.8  gcc-8.1.9  gcc-8.2.0  gcc-8.2.1  gcc-8.2.2  gcc-8.2.3  gcc-8.2.4  gcc-8.2.5  gcc-8.2.6  gcc-8.2.7  gcc-8.2.8  gcc-8.2.9  gcc-8.3.0  gcc-8.3.1  gcc-8.3.2  gcc-8.3.3  gcc-8.3.4  gcc-8.3.5  gcc-8.3.6  gcc-8.3.7  gcc-8.3.8  gcc-8.3.9  gcc-8.4.0  gcc-8.4.1  gcc-8.4.2  gcc-8.4.3  gcc-8.4.4  gcc-8.4.5  gcc-8.4.6  gcc-8.4.7  gcc-8.4.8  gcc-8.4.9  gcc-8.5.0  gcc-8.5.1  gcc-8.5.2  gcc-8.5.3  gcc-8.5.4  gcc-8.5.5  gcc-8.5.6  gcc-8.5.7  gcc-8.5.8  gcc-8.5.9  gcc-8.6.0  gcc-8.6.1  gcc-8.6.2  gcc-8.6.3  gcc-8.6.4  gcc-8.6.5  gcc-8.6.6  gcc-8.6.7  gcc-8.6.8  gcc-8.6.9  gcc-8.7.0  gcc-8.7.1  gcc-8.7.2  gcc-8.7.3  gcc-8.7.4  gcc-8.7.5  gcc-8.7.6  gcc-8.7.7  gcc-8.7.8  gcc-8.7.9  gcc-8.8.0  gcc-8.8.1  gcc-8.8.2  gcc-8.8.3  gcc-8.8.4  gcc-8.8.5  gcc-8.8.6  gcc-8.8.7  gcc-8.8.8  gcc-8.8.9  gcc-8.9.0  gcc-8.9.1  gcc-8.9.2  gcc-8.9.3  gcc-8.9.4  gcc-8.9.5  gcc-8.9.6  gcc-8.9.7  gcc-8.9.8  gcc-8.9.9  gcc-9.0.0  gcc-9.0.1  gcc-9.0.2  gcc-9.0.3  gcc-9.0.4  gcc-9.0.5  gcc-9.0.6  gcc-9.0.7  gcc-9.0.8  gcc-9.0.9  gcc-9.1.0  gcc-9.1.1  gcc-9.1.2  gcc-9.1.3  gcc-9.1.4  gcc-9.1.5  gcc-9.1.6  gcc-9.1.7  gcc-9.1.8  gcc-9.1.9  gcc-9.2.0  gcc-9.2.1  gcc-9.2.2  gcc-9.2.3  gcc-9.2.4  gcc-9.2.5  gcc-9.2.6  gcc-9.2.7  gcc-9.2.8  gcc-9.2.9  gcc-9.3.0  gcc-9.3.1  gcc-9.3.2  gcc-9.3.3  gcc-9.3.4  gcc-9.3.5  gcc-9.3.6  gcc-9.3.7  gcc-9.3.8  gcc-9.3.9  gcc-9.4.0  gcc-9.4.1  gcc-9.4.2  gcc-9.4.3  gcc-9.4.4  gcc-9.4.5  gcc-9.4.6  gcc-9.4.7  gcc-9.4.8  gcc-9.4.9  gcc-9.5.0  gcc-9.5.1  gcc-9.5.2  gcc-9.5.3  gcc-9.5.4  gcc-9.5.5  gcc-9.5.6  gcc-9.5.7  gcc-9.5.8  gcc-9.5.9  gcc-9.6.0  gcc-9.6.1  gcc-9.6.2  gcc-9.6.3  gcc-9.6.4  gcc-9.6.5  gcc-9.6.6  gcc-9.6.7  gcc-9.6.8  gcc-9.6.9  gcc-9.7.0  gcc-9.7.1  gcc-9.7.2  gcc-9.7.3  gcc-9.7.4  gcc-9.7.5  gcc-9.7.6  gcc-9.7.7  gcc-9.7.8  gcc-9.7.9  gcc-9.8.0  gcc-9.8.1  gcc-9.8.2  gcc-9.8.3  gcc-9.8.4  gcc-9.8.5  gcc-9.8.6  gcc-9.8.7  gcc-9.8.8  gcc-9.8.9  gcc-9.9.0  gcc-9.9.1  gcc-9.9.2  gcc-9.9.3  gcc-9.9.4  gcc-9.9.5  gcc-9.9.6  gcc-9.9.7  gcc-9.9.8  gcc-9.9.9  gcc-10.0.0  gcc-10.0.1  gcc-10.0.2  gcc-10.0.3  gcc-10.0.4  gcc-10.0.5  gcc-10.0.6  gcc-10.0.7  gcc-10.0.8  gcc-10.0.9  gcc-10.1.0  gcc-10.1.1  gcc-10.1.2  gcc-10.1.3  gcc-10.1.4  gcc-10.1.5  gcc-10.1.6  gcc-10.1.7  gcc-10.1.8  gcc-10.1.9  gcc-10.2.0  gcc-10.2.1  gcc-10.2.2  gcc-10.2.3  gcc-10.2.4  gcc-10.2.5  gcc-10.2.6  gcc-10.2.7  gcc-10.2.8  gcc-10.2.9  gcc-10.3.0  gcc-10.3.1  gcc-10.3.2  gcc-10.3.3  gcc-10.3.4  gcc-10.3.5  gcc-10.3.6  gcc-10.3.7  gcc-10.3.8  gcc-10.3.9  gcc-10.4.0  gcc-10.4.1  gcc-10.4.2  gcc-10.4.3  gcc-10.4.4  gcc-10
```

```

keviye@localhost:~/scheduling~/userbin/simpriority.c
simrbate -ackbe to
in main()
{
    int n, completed = 0, time = 5;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct tti { int id, priority, done; };
    struct tti *ttis;
    float avgct = 0, avgpt = 0;
    // sort
    for (int i = 0; i < n; i++)
    {
        printf("Enter ID, ID and priority for this task: ");
        scanf("%d %d %d", &ttis[i].id, &ttis[i].priority, &ttis[i].done);
        ttis[i].done = 0;
    }
    // Scheduling logic
    while (completed < n)
    {
        int i = 0;
        int avgpriority = 1000;
        for (int i = 0; i < n; i++)
        {
            if (ttis[i].done == 0)
            {
                if (ttis[i].priority < avgpriority)
                {
                    avgpriority = ttis[i].priority;
                    i = i;
                }
            }
        }
        if (i != -1)
        {
            ttis[i].done = 1;
            completed++;
        }
        else
        {
            return; // All tasks
        }
    }
    // Separate two, for average calculation
    for (int i = 0; i < n; i++)
    {
        avgct += ttis[i].done;
        avgpt += ttis[i].priority;
    }
    printf("Average Turnaround Time: %d\n", avgct / n);
    printf("Average Waiting Time: %d\n", avgpt / n);
}

```

[illegible]

ROUNDROBIN ALGORITHM

```
kaviya@localhost:~$ ls
kaviya@localhost:~$ cat test.c
// test.c
// Author: Kaviya
// Date: 10/10/2023
// Description: A C program to calculate the average waiting time for a set of processes.
// The program uses a round-robin scheduling algorithm with a time quantum of 1 unit.
// The processes are: p1, p2, p3, p4, p5, p6, p7, p8, p9, p10.
// The arrival times are: p1=0, p2=1, p3=2, p4=3, p5=4, p6=5, p7=6, p8=7, p9=8, p10=9.
// The output is the average waiting time.

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define N 10
#define Q 1

// Process structure
struct process {
    int pid;
    int arrival_time;
    int remaining_time;
    int start_time;
    int completion_time;
    int waiting_time;
};

// Global variables
int n = N;
int q = Q;
struct process p[N];
int ready_queue[N];
int ready_queue_size = 0;
int current_time = 0;
int total_waiting_time = 0;

// Function to add a process to the ready queue
void add_to_ready_queue(struct process p) {
    ready_queue[ready_queue_size] = p;
    ready_queue_size++;
}

// Function to remove a process from the ready queue
struct process* remove_from_ready_queue() {
    struct process* p = &ready_queue[0];
    for (int i = 1; i < ready_queue_size; i++) {
        if (ready_queue[i].arrival_time < ready_queue[0].arrival_time) {
            p = &ready_queue[i];
        }
    }
    return p;
}

// Function to calculate the average waiting time
double calculate_average_waiting_time() {
    for (int i = 0; i < n; i++) {
        p[i].remaining_time = p[i].arrival_time;
        p[i].start_time = 0;
        p[i].completion_time = 0;
        p[i].waiting_time = 0;
    }

    while (ready_queue_size > 0) {
        struct process* p = remove_from_ready_queue();
        p->remaining_time -= q;
        p->start_time = current_time;
        current_time += q;
        if (p->remaining_time <= 0) {
            p->completion_time = current_time;
            p->waiting_time = p->start_time - p->arrival_time;
            total_waiting_time += p->waiting_time;
        } else {
            add_to_ready_queue(*p);
        }
    }

    return (double)total_waiting_time / n;
}

int main() {
    // Add processes to the ready queue
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        p[i].arrival_time = i;
        add_to_ready_queue(p[i]);
    }

    // Calculate the average waiting time
    double avg_waiting_time = calculate_average_waiting_time();

    printf("Average Waiting Time: %.2f\n", avg_waiting_time);
    return 0;
}
```

```
kaviye@localhost: --scheduling
kaviye@localhost:~$ ls
bin ch  src  m4sort  headllst  rfiles.sh  memory.pgo  processes  adddfs  set  saepla.sh  scheduling.s0  spstae.lafe  rfare:
der.sh  wroa.sh  heastact  coole  loepe.sk  0aeft.in  persng.ch  srs.lst  semple.c  seopla:3ot  saript  s0  Fongterss  antte.sh
kaviye@localhost:~$ cd scheduling3 />a.cof
kaviye@localhost:~$ scheduling.s0
mmms: R,f,r,r, proriogss: sgt t
kaviye@localhost:~$ scheduling 3 vl rr.c
kaviye@localhost:~$ scheduling 3
kaviye@localhost:~$ scheduling 3 vi +r.c
kaviye@localhost:~$ scheduling 3 .20.864
Enter nuaber of processes:
8
Enter Time Quantum:
5
Enter 2T and 8T for P1:
0.3
Enter 2T and 8T for P3:
1.4
Enter 2T and 8T for P3:
7.2
Enter 2T and 8T for P4:
0.1
Average Tarnaround Tlee: 7.25
Average Marking Ties: 5.25
```